

Large Language Models - Exam

Politecnico di Torino

11 Febbraio 2025

Tempo impiegato: 1 ora 15 min. **Valutazione:** 10,38 / 18,00

Domanda 1 (2 points (no penalty for a wrong answer))

What is the likely output of the following code snippet?

```
1 from transformers import AutoModel, AutoTokenizer
2
3 model_name = "bert-base-uncased"
4 model = AutoModel.from_pretrained(model_name)
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7 prompt = "The result of the operation 2+2 is "
8 tokens = tokenizer(prompt, return_tensors="pt")
9
10 result = model.generate(**tokens)
11 print(tokenizer.batch_decode(result))
12
```

Provide a brief explanation (max 50 words). If an error occurs, indicate the line number and briefly explain why.

Domanda 2 (1 point (-25% penalty for each wrong answer))

What is the purpose of reflection in an agentic system?

- a. To eliminate the need for structured workflows
 - b. To generate human-like responses using predefined scripts
 - c. To store knowledge permanently in short-term memory
 - d. To execute tasks without modifying strategies
 - e. To evaluate past decisions and improve future performance
-

Domanda 3 (1 point (-25% penalty for a wrong answer))

What is the main difference between encoder-decoder cross-attention and encoder self-attention?

- a. In encoder-decoder cross-attention, the decoder attends only to the current token in the input sequence, while in encoder self-attention, the encoder attends to all tokens in the input

- b. Encoder self-attention attends to the encoder's output, while encoder-decoder cross-attention attends to the decoder's previous tokens
 - c. In encoder self-attention, the decoder attends to the input sequence, whereas in encoder-decoder cross-attention, the encoder attends to the output sequence
 - d. Encoder self-attention is used in the decoder, and encoder-decoder cross-attention is used in the encoder
 - e. None of the other options is correct
 - f. Encoder-decoder cross-attention attends only to the previous tokens in the output sequence, while encoder self-attention attends only to tokens in the input sequence
 - g. Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder's output
-

Domanda 4 (1.5 points (no penalty for a wrong answer))

You are given the following sequence of characters:

aceaceaaaaacb

Initially, each character is a separate token. Apply Byte-Pair Encoding to produce 3 additional tokens. What are the generated tokens?

Write each token, in the correct order, in the following slots. Separate the merged tokens with a comma (and no spaces).

Examples:

- if the merged tokens are aa and b, write: aa,b
 - if the merged tokens are x and y, write: x,y
-

Domanda 5 (1 point (-25% penalty for a wrong answer))

Which of the following best describes how BERTScore evaluates text similarity?

- a. It measures sequence similarity using n-gram co-occurrence weighted by IDF scores
- b. It uses a fine-tuned BERT model to classify whether the reference and candidate texts are semantically related or not
- c. It predicts a similarity score by treating the evaluation task as a masked language modeling problem, where BERT predicts the score as the next token
- d. It assigns similarity scores based on syntactic parse trees rather than token embeddings
- e. It uses cosine similarity on static word embeddings trained with Word2Vec or GloVe
- f. It applies a bag-of-words model to compare token frequency distributions between reference and candidate texts
- g. It computes token-level similarity by aligning contextualized embeddings from a pre-trained transformer model and aggregating scores

Domanda 6 (2 points (no penalty for a wrong answer))

Apply absmax quantization to the following values x_1, x_2, x_3, x_4 , to map them to int8.

$$x_1 = -5, \quad x_2 = 4, \quad x_3 = -1, \quad x_4 = 0$$

Write the quantized versions.

Domanda 7 (1 point (-25% penalty for each wrong answer))

Which of the following are benefits of using open-source datasets for training LLMs?

- a. Authenticity and credibility of real-world data
 - b. Guaranteed high accuracy in model predictions
 - c. Ability to acquire timely data and share data freely
 - d. Elimination of bias in training data
 - e. LLMs no longer require fine-tuning
-

Domanda 8 (3 points (no penalty for wrong answers))

Consider a scenario where you have:

- A natural language description of a function's requirements (REQS)
- A set of test cases (TESTS) specifying expected inputs and outputs

You need to design an agentic architecture using LLAMA-based agents to:

- Generate an initial implementation of the function based on REQS
- Refine the code iteratively until all test cases pass

Describe the architecture of LLAMA agents that perform these steps. For each agent:

- Define its role in the pipeline
- Specify its prompt, including:
 - System message: Defines the agent's behavior
 - Context: Information given to the agent from previous steps
 - Possible inputs: Data received from prior agents
 - Expected output: The structured result the agent should return

Notice: It is not necessary to write Python calls to transformers, code for output parsing or actual code for test execution and parsing. Focus on the agentic workflow and prompt engineering techniques.

Domanda 9 (2 points (no penalty for wrong answers))

Consider the following cosine similarity table. Which of the following threshold is more dependable in terms of TPR and FPR? $T = 0.70, 0.80, 0.90$

	Exp1	Exp2	Exp3	Exp4
Act1	0,71	0,65	0,62	0,30
Act2	0,71	0,74	0,30	0,20
Act3	0,54	0,05	0,60	0,55
Act4	0,80	0,40	0,15	0,92

(Note: "Exp" likely refers to Expected/True class or Experiment, and "Act" to Actual/Predicted, or vice-versa. Based on typical confusion matrix problems, assume diagonals are True Positives).

Domanda 10 (1 point (-25% penalty for each wrong answer))

Which solutions can help address the challenge of variation in extracted requirements?

- a. Automatically rejecting extracted requirements with different phrasing
- b. Ignoring minor variations and focusing only on exact matches
- c. Using predefined synonym lists to map similar terms
- d. Using word embeddings to detect semantically similar words
- e. Manually reviewing every extracted requirement

Domanda 11 (1.5 points (-25% penalty for a wrong answer))

The following reward can be used when fine-tuning a model on human feedback.

$$R(x, y) = r_{\theta}(x, y) - \beta \log \frac{\pi_{new}(y|x)}{\pi_{old}(y|x)}$$

Where r_{θ} is the output of a Reward Model, π_{new} and π_{old} are the new and the original policies, respectively (you can see them as the probabilities assigned by the new and original models).

What is the purpose of the term $\log \frac{\pi_{new}(y|x)}{\pi_{old}(y|x)}$?

- a. it increases the entropy of the output (i.e., it helps the model produce more "creative" answers)
- b. it improves the model's capability of following instructions
- c. it prevents the model from collapsing into a model that always predicts a single token
- d. it helps make the model more deterministic
- e. none of the other answers is correct
- f. it prevents the new model from producing outputs that are too dissimilar from the original one
- g. it rewards the model when it generates an answer that humans will "like"

Domanda 12 (1 point (-25% penalty for each wrong answer))

What is "priming" in the context of prompt engineering?

- a. Using reinforcement learning to fine-tune the model
 - b. Pre-training the model with additional data
 - c. Providing initial input to guide the model's response
 - d. Adding noise to the input to test model robustness
 - e. Resetting the model before every new prompt
-

Soluzioni

Domanda 1:

- The code produces an error because `AutoModel` (base BERT) does not support `.generate()`. BERT is an encoder-only model, not built to produce new tokens autoregressively.

Domanda 2:

- (e) To evaluate past decisions and improve future performance.

Domanda 3:

- (g) Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder's output.

Domanda 4:

- 1st merge: a, a
- 2nd merge: a, c
- 3rd merge: ac, e

Domanda 5:

- (g) It computes token-level similarity by aligning contextualized embeddings from a pre-trained transformer model and aggregating scores.

Domanda 6:

- $\text{quantized}(x_1) = -127$
- $\text{quantized}(x_2) = 102$
- $\text{quantized}(x_3) = -25$
- $\text{quantized}(x_4) = 0$

Domanda 7:

- Authenticity and credibility of real-world data
- Ability to acquire timely data and share data freely

Domanda 8: (Summary of architecture)

- **Agent 1: Coder/Implementer.** Role: Generate initial code. System: Expert Python developer. Input: REQS. Output: Python code block.
- **Agent 2: Tester/Refiner.** Role: Run tests and refine code. System: Expert debugger. Input: Code + Test Results. Output: Refined code.
- *Workflow:* Iterative loop where Agent 2 receives feedback (failed tests) and improves the code until all tests pass.

Domanda 9:

- The answer is $T = 0.7$.
- *Reasoning (from student attempt):* For $T = 0.7$, TP=3, FP=1, TN=10, FN=2. TPR=0.6, FPR=0.09.

Domanda 10:

- Using predefined synonym lists to map similar terms.
- Using word embeddings to detect semantically similar words.

Domanda 11:

- (f) it prevents the new model from producing outputs that are too dissimilar from the original one.

Domanda 12:

- (c) Providing initial input to guide the model's response.